

Sistema de Cálculo Mental - DESAFÍO MENTAL

Documento de especificación funcional y técnica del proyecto móvil desarrollado en React Native. La aplicación fue concebida como una plataforma interactiva orientada al entrenamiento cognitivo, cálculo mental, velocidad de reacción y razonamiento matemático bajo presión temporal. El sistema implementa múltiples modos de juego, generación dinámica de operaciones, gestión de sesiones, persistencia local y estadísticas históricas.

1. Objetivo General

Diseñar una aplicación móvil capaz de generar desafíos matemáticos dinámicos orientados a mejorar habilidades de cálculo mental mediante mecánicas de videojuego. El objetivo principal consiste en mantener sesiones cortas, rápidas y progresivas utilizando sistemas de dificultad, métricas de desempeño y retroalimentación visual.

2. Objetivos Específicos

- Generar operaciones matemáticas dinámicamente.
- Registrar tiempos de respuesta.
- Aplicar un algoritmo de puntuación basado en desempeño.
- Implementar progresión por niveles.
- Registrar estadísticas históricas.
- Reiniciar automáticamente sesiones y estados inconsistentes.
- Implementar arquitectura desacoplada reutilizable mediante hooks.

3. Arquitectura General

El proyecto utiliza una arquitectura basada en componentes reutilizables y hooks personalizados. Capas principales: **UI**: pantallas React Native, componentes visuales y navegación.

Hooks: lógica central reutilizable.

Math: generación dinámica de ecuaciones.

Storage: persistencia local AsyncStorage.

Scoring: algoritmo de puntuación y métricas.

Estructura: app/

components/

hooks/

lib/math

lib/storage

lib/scoring

4. Generación Dinámica de Operaciones

Las operaciones no se almacenan estáticamente. Se generan en tiempo real utilizando un generador central. Archivo principal: lib/math/generator.ts Dificultades: Nivel 1: • sumas

- restas
- resultados positivos

Nivel 2: • suma

- resta
- multiplicación
- división simple

Nivel 3: • operaciones combinadas

- múltiples términos
- dificultad incremental

5. Sistema de Juego General

Se implementó un hook reutilizable denominado useGame() responsable de: • temporizador

- progreso de ronda
- navegación
- cálculo de estadísticas
- transición entre preguntas
- persistencia final
- manejo del ciclo de vida

Todos los modos heredan comportamiento desde este hook.

6. Modo Clásico

El usuario ingresa manualmente respuestas utilizando un teclado numérico propio. Flujo: Generar operación → ingresar respuesta → validar → calcular puntaje → siguiente ronda Características: • 10 interacciones por nivel

- 15 segundos por operación
- teclado numérico personalizado
- pantalla resumen
- progresión automática

7. Modo Verdadero/Falso

Se genera una ecuación y un resultado potencialmente incorrecto. Ejemplo: $24 + 18 = 53$ El usuario debe determinar: ✓ Verdadero X Falso El sistema genera errores cercanos al resultado real para evitar respuestas triviales.

8. Modo Multiple Choice

Se generan cuatro opciones posibles: 1 correcta 3 distractores Distractores: • valores cercanos
• errores de cálculo frecuentes
• variaciones pequeñas Esto obliga al jugador a resolver mentalmente la operación.

9. Sistema de Puntaje

Reglas implementadas: Respuesta correcta rápida (<75% tiempo disponible) +100 Respuesta correcta +70 Incorrecta -30 Sin respuesta -50 Puntaje mínimo para aprobar: 760 puntos

10. Sistema de Niveles

Nivel1 → Nivel2 → Nivel3 Reglas: • aprobación secuencial
• perder nivel 2 o 3 reinicia desde nivel1
• acumulación entre niveles únicamente durante la sesión actual

11. Persistencia

La aplicación utiliza AsyncStorage. Información almacenada: • score
• tiempo promedio
• fecha
• nivel alcanzado
• modo jugado
• correctas/incorrectas

12. Sistema de Estadísticas

Las estadísticas se encuentran separadas por modo. Se implementó un carrusel con: Clásico Verdadero/Falso Multiple Choice Cada vista contiene: • mejor puntaje
• tiempo promedio
• ranking histórico
• fecha y hora

13. Tecnologías

Frontend: React Native Expo Router TypeScript Persistencia: AsyncStorage Animaciones: Animated API Arquitectura: Hooks reutilizables